

Vim

Vim

Vim - 无处不在的文本编辑器。

简介

Vim 是从 vi 发展出来的一个文本编辑器。其代码补完、编译及错误跳转等方便编程的功能特别丰富，在程序员群体中被广泛使用。

安装

Linux 系统通常自带 Vim，打开终端输入 `vim` 即可启用。

若需手动安装，Vim 的 [官方网站](#) 提供了下载的 [说明文档](#)，按照需求编译安装即可。

Vim 的模式与常用键位

Vim 的基础操作在 Vim 自带的教程里将会讲述。打开终端输入 `vimtutor` 即可进入教程。

这些操作通常需要二三十分钟来大致熟悉。

命令模式 (Command Mode)

进入 Vim 后的默认模式。

此状态下敲击键盘动作会被 Vim 识别为命令，而非输入字符，比如我们此时按下 `i`，并不会输入一个字符，`i` 被当作了一个命令。

Vim 的方向键是 `↑`、`↓`、`←`、`→`，或者 `h`、`j`、`k`、`l`。

```
1          ↑ (k)
2          ^
3 (h)← <      > → (l)
4          v
5          ↓ (j)
```

以下是命令模式常用的命令：

- `i` 切换到输入模式，在光标当前位置开始输入文本。按 `Esc` 键可回到普通模式。
- `x` 用于删除光标后的一个字符。
- `:` 切换到底线命令模式，以在最底一行输入命令。
- `a` 切换到输入模式，在光标后开始输入文本。
- `o` 切换到输入模式，在光标下插入新的一行；`O` 切换到输入模式，在光标上插入新的一行。
- `p` 粘贴剪贴板内容到光标下方；`P` 粘贴剪贴板内容到光标上方。
- `dd` 删除光标所在的一整行。
- `d` 命令也是删除，通常配合其他键使用。
- `u` 撤销上一次对文本的更改。
- `y` 命令可以复制被选中的区域。需要按 `v` 进入可视模式操作。
- `yy` 复制当前行。
- `Ctrl + r` 重做上次撤销的操作。

- `:w` 保存文件，常配合 `q` 保存退出。
- `:q` 退出 Vim。
- `:q!` 强制退出 Vim，不保存修改。

部分其他命令：

- `c` 命令用于修改，相当于 `di`。
- `=` 命令可以以默认格式对选中行应用自动缩进。
- `==` 自动缩进当前行。
- `.` 命令可以重复上次执行的命令。
- `gg` 命令可跳至代码的开头；`G` 命令可跳至代码最后一行的开头；`G` 命令前加数字可跳至指定行。
- `w` 可以跳到下个单词的开头；`e` 可以跳到当前单词或下一单词的结尾；`b` 可以跳到当前单词或上一单词的的开头；`0` 可以跳至行首；`$` 可以跳至行尾。`w`、`e`、`0`、`$` 还可以与其他命令组合，比如 `de`、`dw`、`d0` 和 `d$` 分别对应删至单词尾、删至下个单词头、删至行首和删至行尾。

命令模式下按 `/`，下方即会出现查找框，输入需要查找的字符，按回车后就能查看搜索结果。如果有多个查找结果，按 `n` 即可跳至下一个查找结果；按 `N` 可跳至上一个。

命令模式下按 `*` 可以查找当前光标下的单词。

在输入某个命令前，输入一个数字 `n` 的话，命令就会重复 `n` 次。

输入模式 (Insert Mode)

在命令模式下按下 `i` 就进入了输入模式，按 `Esc` 键可以返回到命令模式。

在输入模式中，可以使用以下按键：

- 字符按键以及 `Shift` 组合，输入字符
- `ENTER`，回车键，换行
- `BACK SPACE`，退格键，删除光标前一个字符
- `DEL`，删除键，删除光标后一个字符
- 方向键，在文本中移动光标
- `HOME/END`，移动光标到行首/行尾
- `Page Up/Page Down`，上/下翻页
- `Insert`，切换光标为输入/替换模式，光标将变成竖线/下划线
- `ESC`，退出输入模式，切换到命令模式

在输入模式下按 `Ctrl+o` 即可进入「输入 - 命令模式」，执行完一次操作后又会自动回到输入模式。

底线命令行模式

命令模式下按 `:`，进入底线命令行模式。

底线命令行模式可以输入单个或多个字符的命令，可用的命令非常多。

在底线命令行模式中，基本的命令有：

- `:help/:h` 查看英文版 Vim 在线帮助文档。
- `:w` 保存文件。
- `:q` 退出 Vim。
- `:wq` 保存文件，退出 Vim。
- `:q!/:!q` 强制退出 Vim，不保存修改。
- `:e filename` 可以打开当前目录下的指定文件。
- `:s` 命令是替换。

```

1 " 把当前行第一个匹配的 str1 替换成 str2
2 :s/str1/str2/
3 " 把当前行所有的 str1 替换成 str2
4 :s/str1/str2/g
5 " 把当前行所有的 str1 替换成 str2, 在替换前询问
6 :s/str1/str2/gc
7 " 把第 x1 行至 x2 行中, 每一行第一个匹配的 str1 替换成 str2
8 :x1,x2 s/str1/str2/
9 " 把第 x1 行至 x2 行中所有的 str1 替换成 str2
10 :x1,x2 s/str1/str2/g
11 " 第 x1 行至 x2 行中所有的 str1 替换成 str2, 在替换前询问
12 :x1,x2 s/str1/str2/gc
13 " 把所有行第一个匹配的 str1 替换成 str2
14 :%s/str1/str2/
15 " 把全文件所有的 str1 替换成 str2
16 :%s/str1/str2/g
17 " 把全文件所有的 str1 替换成 str2, 在替换前询问
18 :%s/str1/str2/gc

```

如果命令形式是 `:! command`, 则命令将在 `bash` 终端执行。

按 `Esc` 键可以退出底线命令模式。

可视模式 (Visual mode)

按 `v` 进入可视模式, 多用于选中区域。按 `V` (`Shift+v`) 进入行可视模式, 用于选中行。

按 `Ctrl+v` 或 `Ctrl+q` 进入块可视模式 (visual block)。

进入块可视模式后, 按 `I` 或 `A` 进入插入模式 (相当于 `i` 和 `a`) , 退出插入模式后对本行所做的改动将被应用到选中的每一行同一位置。常用于批量添加注释。

选中后输入 `y` 或 `d` 亦可执行相应命令。

三种可视模式可以通过按键相互转化。

Vim 的快捷键

可参考 [史上最全 Vim 快捷键键位图—入门到进阶](#)

进阶知识

. 命令

Vim 的使用者不可避免地会抗拒重复的文本修改, 因为 Vim 注定比其他编辑器会多出两次按键——`Esc`与`i`。但是, Vim 其实提供了重复命令 `.`, 它适用于重复的添加、修改、删除文本操作。

`.` 命令可以重复上次执行的命令。但是这个「命令」并不只限于单一的命令, 它也可以是 数字 + 命令 的组合; 进入插入模式 + 输入文本 + `Esc` 也是命令的一种。所以, 适当使用 `.` 命令才能达到最高的效率。

例如, 如下代码的每一行末尾都少了分号:

```

1 int a, b
2 cin >> a >> b
3 cout << a + b
4 return 0

```

将 `.` 与搭配移动到行尾插入命令 `A` 使用, 就能高效地补上末尾的分号。

```
1 A;<Esc>
2 " 重复下面的命令
3 j.
```

再例如，如下代码中，后面五个赋值语句的数组名全部写错了：

```
1 int check() {
2   book[1] = 1, book[2] = 1, book[3] = 1, bok[1] = 1, bok[2] = 1, bok[3] = 1,
3   bok[4] = 1, bok[5] = 1;
4   return 0;
5 }
```

一个个改过于麻烦，而命令行模式的 `s` 命令又会全部改掉。

第一种改法是搭配普通模式下的 `s` 命令（删除光标处字符并进入插入模式）使用。来到第一个错误的数组名首字母处，按下 `3s/cw`，输入正确的数组名并退出。之后把光标一个个移过去，再使用 `.` 命令。

第二种比较节省时间的改法是利用查找模式修改。键入 `/bok`，接着按下回车，并使用 `n` 键来到第一个错误的数组名首字母处，键入 `3s 新数组名 <Esc>`，最后重复 `n.`。

第三种改法是简易查找命令 `f`。在一行中普通模式下，`f + 单个字符` 即可查找此行中出现的这个字符并将光标移至字符处；按 `;` 查找下一个，`,` 查找上一个。所以对于上面的代码，只需键入 `fb;;;` 之后进入插入模式修改，然后 `;` 即可。这种改法适用于只需行内移动的情况。

宏

Vim 的宏功能可以重复任意长的命令。

使用宏之前要先「录制」，即把一串按键操作录下来再回放，这样就达到了重复的效果。录制的方法很简单，普通模式下键入 `q` 开始录制。下一步，为录制的宏指定一个执行的命令键，可以按下 26 个字母中的任意一个来指定。这时左下方会显示 记录中 @刚刚选择的字母。然后就可以开始录制命令了。同理，普通模式下按 `q` 暂停录制。

使用方法为按下：进入命令行模式，键入 `@选择的记录字母`，然后之前录制的命令就被调用了。

将 `.` 和宏组合，即录制宏 $\rightarrow$ 调用宏 $\rightarrow$ 重复命令 $\rightarrow$ 数字 `+`，可以达到非常高的效率。

normal 命令

该命令与普通模式有关，效果是在指定行重复命令。

按：进入命令行模式，输入如下命令：

```
1 :a,b normal command
```

或者：

```
1 :a,b norm command
```

以上命令的意思是在普通模式下，对 `a~b` 行执行 `command` 命令。

由于 `normal` 命令可以被 `.` 命令重复调用，且其易于理解，它的使用频率甚至更高于宏。

数字 `+` + 宏 + `normal`

以上三种命令可以组合使用。例如：

我下载了一本书，我需要它的每一个章节都变成「标题」，以方便转换成 mobi 之类的格式，或者方便生成 TOC 目录跳转，怎么办呢？

以下是用 Vim 处理的过程：

1. 按下 / 调出查找框，输入正则表达式进行查找；
2. 用 `q` 命令开始录制宏；
3. 键入 `I#` 命令，然后按下 `ESC`；
4. 用 `q` 进行修改操作并结束宏录制；
5. 键入 `normal n@`字母 转到下一处并重复上一步操作；
6. 键入 数字 + . 多次重复。

外部链接

- [Vim 官网](#)
- [原作者提供的配置](#)
- [Vim 调试: termdebug 入门](#)
- [Vim scripting cheatsheet](#)
- [Learn Vimscript the Hard Way](#)
- [Linux vi/vim | 菜鸟教程](#)

本页面最近更新：2024/2/28 18:46:05, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者：[383494](#), [ChungZH](#), [CoderOJ](#), [danielqfmai](#), [Doveqise](#), [Enter-tainer](#), [lr1d](#), [LuoshuiTianyi](#), [NachtgeistW](#), [ouuan](#), [partychicken](#), [SkyeYoung](#), [StudyingFather](#), [SukkaW](#), [Xeonacid](#), [akakw1](#), [Alisahhh](#), [billchenchina](#), [CCXXI](#), [HarryWang29](#), [Kewth](#), [Kewth](#), [ksyx](#), [Marcythm](#), [mgt](#), [Mout-sea](#), [Petalzu](#), [s0cks5](#), [SodaCris](#), [SodaCris](#), [TianyiQ](#), [Tiphereth-A](#), [tLLWtG](#), [Too-Naive](#)

本页面的全部内容 [在 CC BY-SA 4.0 和 SATA 协议之条款下](#) 提供，附加条款亦可能应用